

# Introduction to the Nektar++ framework

***Víctor Ballester Ribó***  
*Imperial College London*

# Agenda

- The **what, where, when, why, and how** Nektar++
- Structure of the code and small tutorial



Nektar++  
www.nektar.info

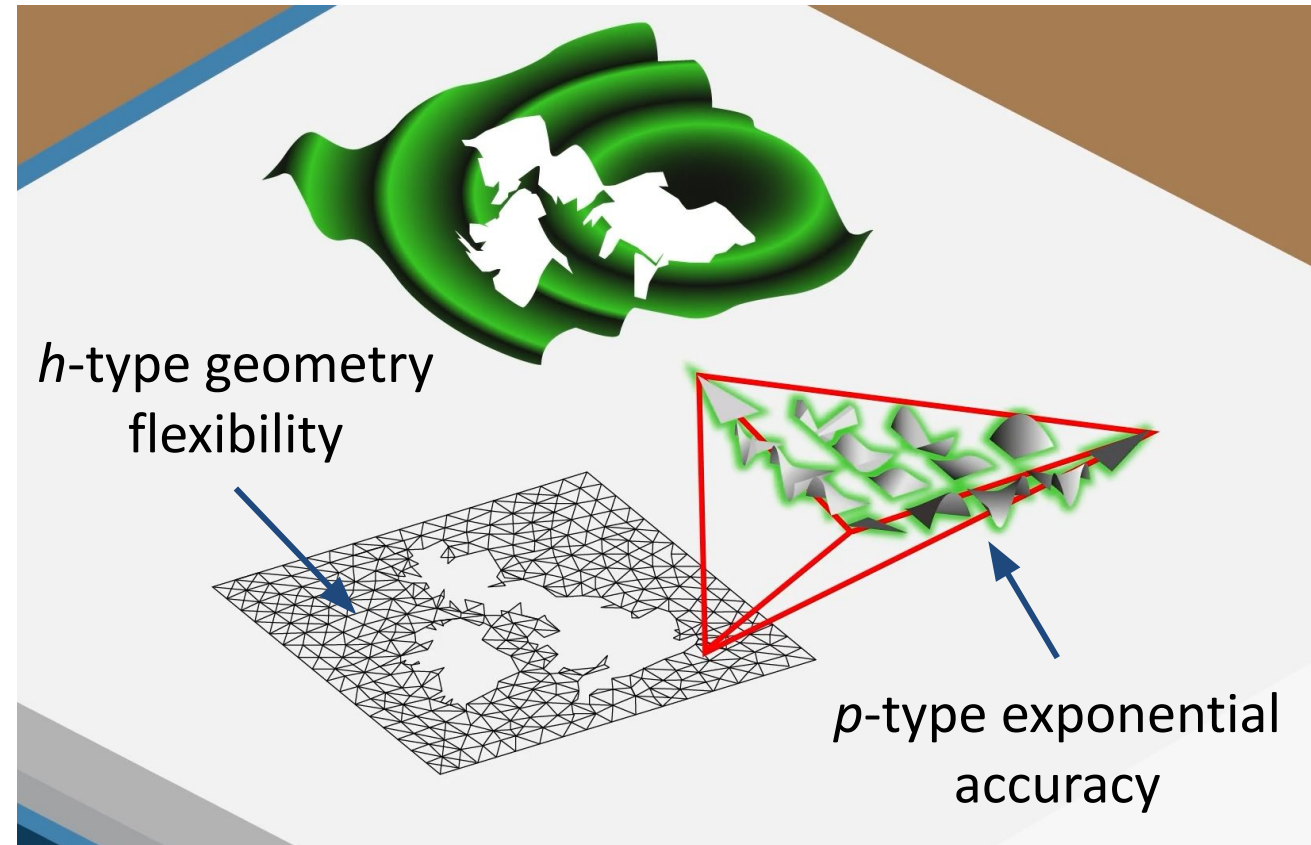
IMPERIAL

Newcastle  
University



# What is Nektar++?

- Open-source framework, to enable the high-fidelity solution of PDEs.
- Based on **spectral-*hp* element method**, a high polynomial order FEM.
- High fidelity simulations!



# What is Nektar++?

- Modern C++
- Python API
- MIT licence
- Includes with several fully fledged Solvers
- Tailored for fluid dynamics problem with DNS and implicit LES for turbulence



- Excellent HPC performance
- Supported on the UK National Supercomputer ARCHER2



- [www.nektar.info](http://www.nektar.info)
- [gitlab.nektar.info](https://gitlab.nektar.info)



# What is Nektar++?

- It has been used by several industries such as Rolls Royce, McLaren, UKAEA
- It is on the flag ship codes of UK Turbulence Consortium (UKTC)

**Rolls  
Royce**



**UK Atomic  
Energy  
Authority**

**McLaren**



# Where and When started Nektar++?



Nektar++ is under active and continuous development since 2005

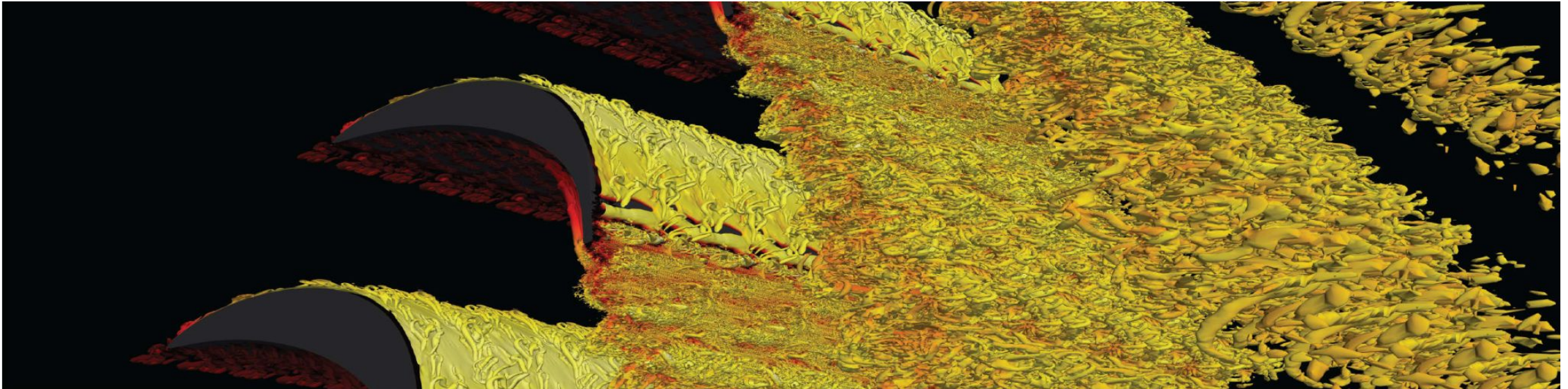
# Where and When started Nektar++?



Nektar++ is under active and continuous development since 2005

# Why Nektar++?

- Cutting-edge technologies such as aeroplanes or wind turbines require high accuracy information regarding turbulent flows.
- Nektar++ provides this high accuracy needed to resolve very finely these problems.



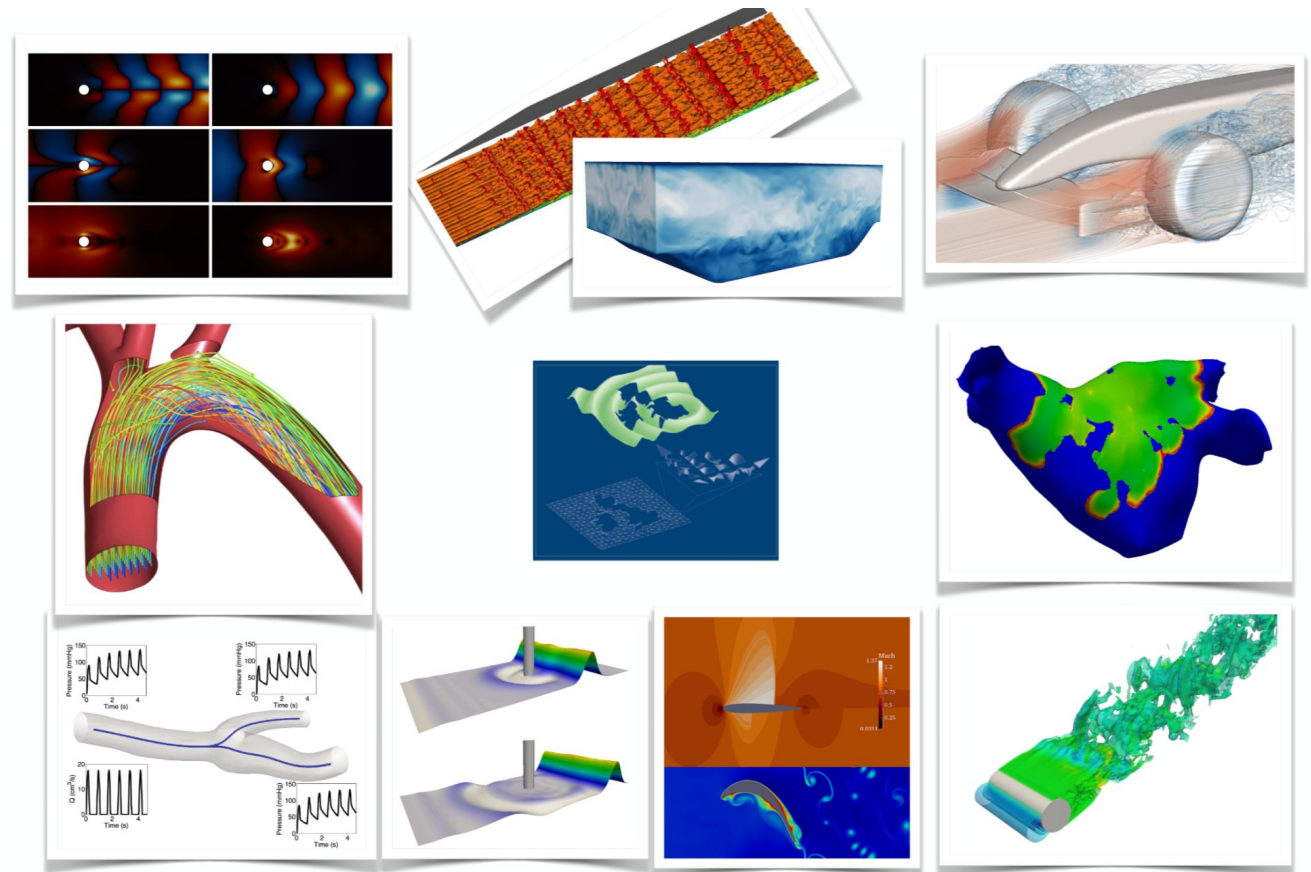
# What's in Nektar++?

- Advection-Diffusion-Reaction Solver
- Acoustic Solver
- Cardiac Electrophysiology Solver
- **Compressible Flow Solver**
- Diffusion Solver
- Images Warping Solver
- **Incompressible NS Solver**
- Linear Elasticity Solver
- Pulsewave Solver
- Shallow Water Solver
- Vortex-Wave Solver



Nektar++  
www.nektar.info

IMPERIAL



# Agenda

- The **what, where, when, why, and how** Nektar++
- Structure of the code and small tutorial



Nektar++  
[www.nektar.info](http://www.nektar.info)

IMPERIAL

Newcastle  
University



# How to use Nektar++?

Nektar++ requires a session file (XML) to work. It contains:

- *Geometry*: A mesh that represents the domain of interest.
- *Expansions*: Information on the basis to be on each element.
- *Conditions*: Solver-specific information such as BCs and ICs to be used.

```
<NEKTAR>
  <GEOMETRY>
    <!-- Information regarding the mesh goes in this block. -->
  </GEOMETRY>
  <EXPANSIONS>
    <!-- The basis to be used in each element. -->
  </EXPANSIONS>
  <CONDITIONS>
    <!-- Solver-specific information, including boundary and initial conditions. -->
  </CONDITIONS>
</NEKTAR>
```

# How to run Nektar++?

## Serial

```
> $SOLVER_NAME full_session_file.xml
```

```
> $SOLVER_NAME mesh_file.xml session_file.xml
```

## Parallel

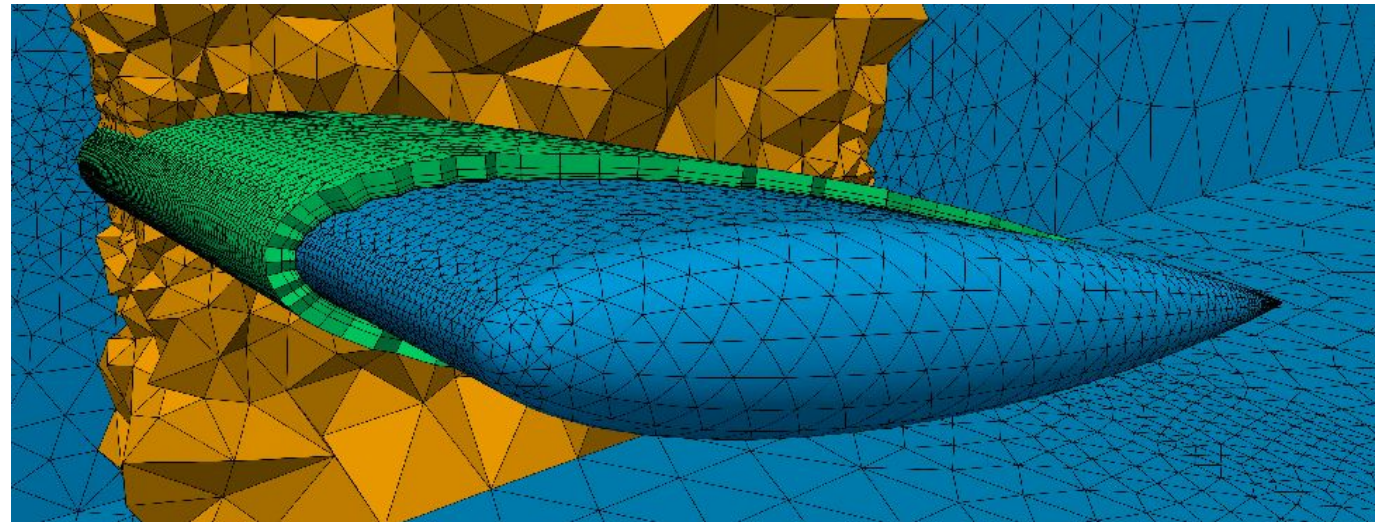
```
> mpirun -np 8 $SOLVER_NAME full_session_file.xml
```

```
> mpirun -np 8 $SOLVER_NAME mesh_file.xml session_file.xml
```

# Defining a mesh

- The inbuilt utility to deal with meshes is **NekMesh** (more on that in the next talk!)
- We usually separate the mesh part from the session file.

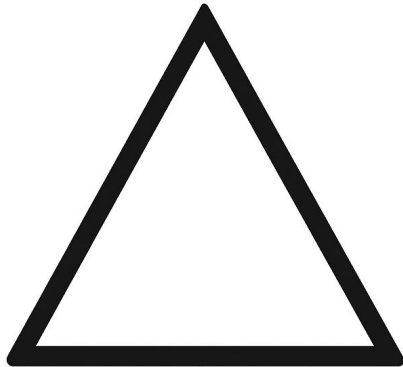
In the tutorials we will provide you with the mesh file!



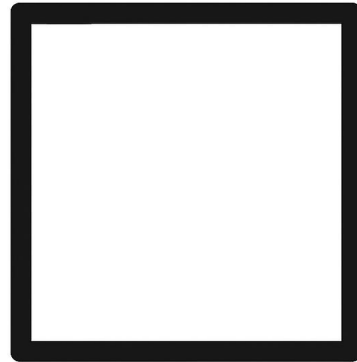
# Types of elements

2D

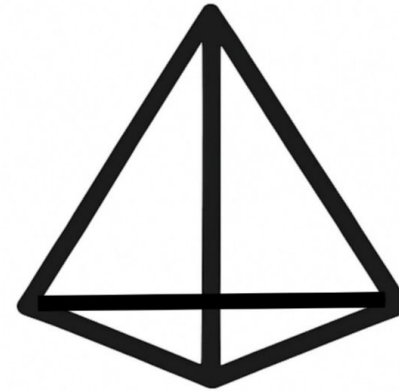
triangle



quadrilateral

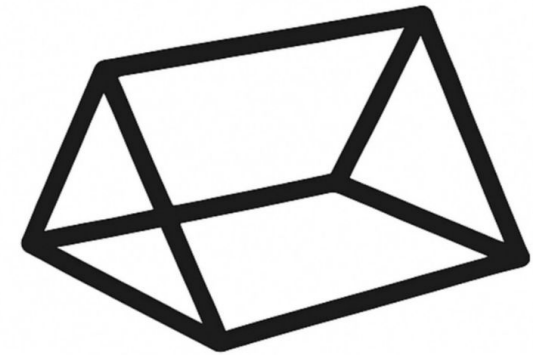


Tetrahedron

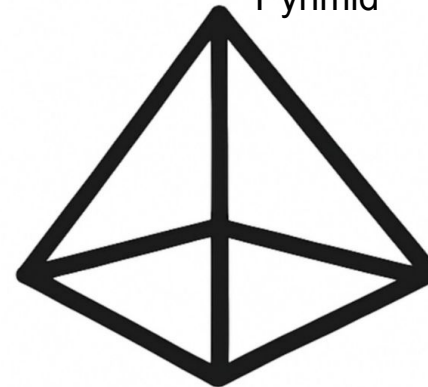


3D

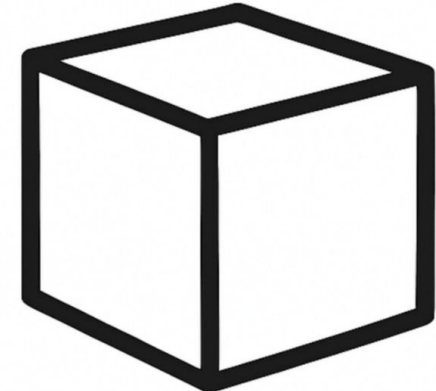
Prism



Pyramid



Hexahedron



... and all these could be curved as well!

# Mesh file

```
1 <?xml version="1.0" encoding="utf-8" ?>
2 <NEKTAR>
3   <GEOMETRY DIM="2" SPACE="2">
4     <VERTEX>
5       <V ID="0">2.00000000e-01 -1.00000000e+00 0.00000000e+00</V>
6       <V ID="1">5.09667784e-01 -6.15240515e-01 0.00000000e+00</V>
7       ...
8       <V ID="68">-1.00000000e+00 1.25000000e-01 0.00000000e+00</V>
9     </VERTEX>
10    <EDGE>
11      <E ID="0"> 0 1 </E>
12      <E ID="1"> 1 2 </E>
13      ...
14      <E ID="153"> 40 68 </E>
15    </EDGE>
16    <ELEMENT>
17      <T ID="0"> 0 1 2 </T>
18      <T ID="1"> 3 4 5 </T>
19      ...
20      <Q ID="85"> 146 93 153 151 </Q>
21    </ELEMENT>
22    <COMPOSITE>
23      <C ID="1"> T[0-30] </C>
24      <C ID="2"> Q[62-85] </C>
25      <C ID="3"> T[31-61] </C>
26      <C ID="100"> E[46,12,20,10,45] </C>
27      <C ID="200"> E[50,32,108,111,114,117,87,103] </C>
28      <C ID="300"> E[100,64,74,66,99] </C>
29      <C ID="400"> E[49,33,148,150,152-153,86,104] </C>
30    </COMPOSITE>
31    <DOMAIN> C[1,2,3] </DOMAIN>
32  </GEOMETRY>
33 </NEKTAR>
```

Vertices

Edges

Triangles

Quadrilaterals

Domain

Boundaries

# Setting up the session file: *Expansions*

- Here we tell Nektar++ which **type** of polynomial do we choose and the maximum order (**nummodes**) of those for each **field** and each part (**composite**) of the mesh.

```
1 <EXPANSIONS>
2   <E COMPOSITE="C[1]" NUMMODES="5" TYPE="MODIFIED" FIELDS="u" />
3   <E COMPOSITE="C[2]" NUMMODES="5" TYPE="MODIFIED" FIELDS="u" />
4   <E COMPOSITE="C[3]" NUMMODES="5" TYPE="MODIFIED" FIELDS="u" />
5 </EXPANSIONS>
```

# Setting up the session file: *Expansions*

- Inf-sup condition for **Incompressible Navier-Stokes simulations**:  
polynomial order pressure = polynomial order velocity - 1
- This ensures existence and uniqueness of solutions.

```
2  <EXPANSIONS>
3  |   <E COMPOSITE="C[100,101,102]" NUMMODES="7" FIELDS="u,v" TYPE="MODIFIED" />
4  |   <E COMPOSITE="C[100,101,102]" NUMMODES="6" FIELDS="p" TYPE="MODIFIEDQUADPLUS1" />
5  </EXPANSIONS>
```

# Setting up the session file: *Conditions*

- In this section we specify all the parameters concerning the equations we want to solve.
- Let's do an example. Consider the following advection problem.

$$\frac{\partial u}{\partial t} + V_x \frac{\partial u}{\partial x} + V_y \frac{\partial u}{\partial y} = 0,$$

$$u(x, y; t = 0) = \sin(\kappa x) \cos(\kappa y),$$

$$u(x_b = [-1, 1], y_b; t) = \text{periodic},$$

$$u(x_b, y_b = [-1, 1]; t) = \sin(\kappa(x - V_x t)) \cos(\kappa(y - V_y t)),$$

# Setting up the session file: *Conditions*

```
1 <CONDITIONS>
2   <PARAMETERS>
3     <P> FinTime = 1.0 </P>
4     <P> TimeStep = 0.001 </P>
5     <P> NumSteps = FinTime/TimeStep </P>
6     <P> IO_CheckSteps = 100 </P>
7     <P> IO_InfoSteps = 100 </P>
8     <P> advx = 2.0 </P>
9     <P> advy = 0.0 </P>
10    <P> k = 2*PI </P>
11  </PARAMETERS>
12
13  <SOLVERINFO>
14    <I PROPERTY="EQTYPE" VALUE="UnsteadyAdvection" />
15    <I PROPERTY="Projection" VALUE="DisContinuous" />
16    <I PROPERTY="AdvectionType" VALUE="WeakDG" />
17    <I PROPERTY="UpwindType" VALUE="Upwind" />
18  </SOLVERINFO>
19
20  <TIMEINTEGRATIONSCHEME>
21    <METHOD> RungeKutta </METHOD>
22    <ORDER> 4 </ORDER>
23  </TIMEINTEGRATIONSCHEME>
24  ...
```

## Parameters

Setting the solution parameters in this section

FinalTime: Final Simulation time

TimeStep: Time-step size

NumStep: Number of steps

We can set TimeStep and NumStep and skip FinalTime

IO\_CheckSteps: For checkpoint output

IO\_CheckSteps=100 means output every 100 steps

IO\_InfoSteps: Print out information every IO\_infoSteps

# Setting up the session file: *Conditions*

```

1 <CONDITIONS>
2   <PARAMETERS>
3     <P> FinTime = 1.0 </P>
4     <P> TimeStep = 0.001 </P>
5     <P> NumSteps = FinTime/TimeStep </P>
6     <P> IO_CheckSteps = 100 </P>
7     <P> IO_InfoSteps = 100 </P>
8     <P> advx = 2.0 </P>
9     <P> advy = 0.0 </P>
10    <P> k = 2*PI </P>
11  </PARAMETERS>
12
13  <SOLVERINFO>
14    <I PROPERTY="EQTYPE" VALUE="UnsteadyAdvection" />
15    <I PROPERTY="Projection" VALUE="DisContinuous" />
16    <I PROPERTY="AdvectionType" VALUE="WeakDG" />
17    <I PROPERTY="UpwindType" VALUE="Upwind" />
18  </SOLVERINFO>
19
20  <TIMEINTEGRATIONSCHEME>
21    <METHOD> RungeKutta </METHOD>
22    <ORDER> 4 </ORDER>
23  </TIMEINTEGRATIONSCHEME>
24  ...

```

## Solver info

Setting the specific settings for the solver

You'll find all possible  
values in the docs!

$$\frac{\partial u}{\partial t} + V_x \frac{\partial u}{\partial x} + V_y \frac{\partial u}{\partial y} = 0,$$

$$u(x, y; t = 0) = \sin(\kappa x) \cos(\kappa y),$$

$$u(x_b = [-1, 1], y_b; t) = \text{periodic},$$

$$u(x_b, y_b = [-1, 1]; t) = \sin(\kappa(x - V_x t)) \cos(\kappa(y - V_y t)),$$

# Setting up the session file: *Conditions*

```

1 <CONDITIONS>
2   <PARAMETERS>
3     <P> FinTime = 1.0 </P>
4     <P> TimeStep = 0.001 </P>
5     <P> NumSteps = FinTime/TimeStep </P>
6     <P> IO_CheckSteps = 100 </P>
7     <P> IO_InfoSteps = 100 </P>
8     <P> advx = 2.0 </P>
9     <P> advy = 0.0 </P>
10    <P> k = 2*PI </P>
11  </PARAMETERS>
12
13  <SOLVERINFO>
14    <I PROPERTY="EQTYPE" VALUE="UnsteadyAdvection" />
15    <I PROPERTY="Projection" VALUE="DisContinuous" />
16    <I PROPERTY="AdvectionType" VALUE="WeakDG" />
17    <I PROPERTY="UpwindType" VALUE="Upwind" />
18  </SOLVERINFO>
19
20  <TIMEINTEGRATIONScheme>
21    <METHOD> RungeKutta </METHOD>
22    <ORDER> 4 </ORDER>
23  </TIMEINTEGRATIONScheme>
24  ...

```

## Time Integration

Time Integration can be set directly in the solverInfo as below

```
<I PROPERTY="TimeIntegrationMethod" VALUE="IMEXOrder2" />
```

Or with more details in a separate tags

Example of supported options  
IMEX : Implicit-explicit  
RungeKutta

$$\frac{\partial u}{\partial t} + V_x \frac{\partial u}{\partial x} + V_y \frac{\partial u}{\partial y} = 0,$$

$$u(x, y; t = 0) = \sin(\kappa x) \cos(\kappa y),$$

$$u(x_b = [-1, 1], y_b; t) = \text{periodic},$$

$$u(x_b, y_b = [-1, 1]; t) = \sin(\kappa(x - V_x t)) \cos(\kappa(y - V_y t)),$$

# Setting up the session file: *Conditions*

## Boundary Conditions

```

4  ...
5  <VARIABLES>
6    <V ID="0"> u </V>
7  </VARIABLES>
8
9  <BOUNDARYREGIONS>
10    <B ID="0"> C[100] </B>
11    <B ID="1"> C[200] </B>
12    <B ID="2"> C[300] </B>
13    <B ID="3"> C[400] </B>
14  </BOUNDARYREGIONS>
15
16  <BOUNDARYCONDITIONS>
17    <REGION REF="0">
18      <D VAR="u" USERDEFINEDTYPE="TimeDependent"
19        VALUE="sin(k*(x-advx*t))*cos(k*(y-advy*t))" />
20    </REGION>
21    <REGION REF="1">
22      <P VAR="u" VALUE="[3]" />
23    </REGION>
24    <REGION REF="2">
25      <D VAR="u" USERDEFINEDTYPE="TimeDependent"
26        VALUE="sin(k*(x-advx*t))*cos(k*(y-advy*t))" />
27    </REGION>
28    <REGION REF="3">
29      <P VAR="u" VALUE="[1]" />
30    </REGION>
31  </BOUNDARYCONDITIONS>
32
33  <FUNCTION NAME="InitialConditions">
34    <E VAR="u" VALUE="sin(k*x)*cos(k*y)" />
35  </FUNCTION>
36
37  <FUNCTION NAME="AdvectionVelocity">
38    <E VAR="Vx" VALUE="advx" />
39    <E VAR="Vy" VALUE="advy" />
40  </FUNCTION>
45 </CONDITIONS>

```

$$\frac{\partial u}{\partial t} + V_x \frac{\partial u}{\partial x} + V_y \frac{\partial u}{\partial y} = 0,$$

$$u(x, y; t = 0) = \sin(\kappa x) \cos(\kappa y),$$

$$u(x_b = [-1, 1], y_b; t) = \text{periodic},$$

$$u(x_b, y_b = [-1, 1]; t) = \sin(\kappa(x - V_x t)) \cos(\kappa(y - V_y t)),$$

# Setting up the session file: *Conditions*

## Initial condition

```

4  ...
5  <VARIABLES>
6    <V ID="0"> u </V>
7  </VARIABLES>
8
9  <BOUNDARYREGIONS>
10    <B ID="0"> C[100] </B>
11    <B ID="1"> C[200] </B>
12    <B ID="2"> C[300] </B>
13    <B ID="3"> C[400] </B>
14  </BOUNDARYREGIONS>
15
16  <BOUNDARYCONDITIONS>
17    <REGION REF="0">
18      <D VAR="u" USERDEFINEDTYPE="TimeDependent"
19        VALUE="sin(k*(x-advx*t))*cos(k*(y-advy*t))" />
20    </REGION>
21    <REGION REF="1">
22      <P VAR="u" VALUE="[3]" />
23    </REGION>
24    <REGION REF="2">
25      <D VAR="u" USERDEFINEDTYPE="TimeDependent"
26        VALUE="sin(k*(x-advx*t))*cos(k*(y-advy*t))" />
27    </REGION>
28    <REGION REF="3">
29      <P VAR="u" VALUE="[1]" />
30    </REGION>
31  </BOUNDARYCONDITIONS>
32
33  <FUNCTION NAME="InitialConditions">
34    <E VAR="u" VALUE="sin(k*x)*cos(k*y)" />
35  </FUNCTION>
36
37  <FUNCTION NAME="AdvectionVelocity">
38    <E VAR="Vx" VALUE="advx" />
39    <E VAR="Vy" VALUE="advy" />
40  </FUNCTION>
45 </CONDITIONS>

```

$$\frac{\partial u}{\partial t} + V_x \frac{\partial u}{\partial x} + V_y \frac{\partial u}{\partial y} = 0,$$

$$u(x, y; t = 0) = \sin(\kappa x) \cos(\kappa y),$$

$$u(x_b = [-1, 1], y_b; t) = \text{periodic},$$

$$u(x_b, y_b = [-1, 1]; t) = \sin(\kappa(x - V_x t)) \cos(\kappa(y - V_y t)),$$

# Running Nektar++

1. We run with: 

```
> ADRSolver ADR_mesh.xml ADR_conditions.xml
```

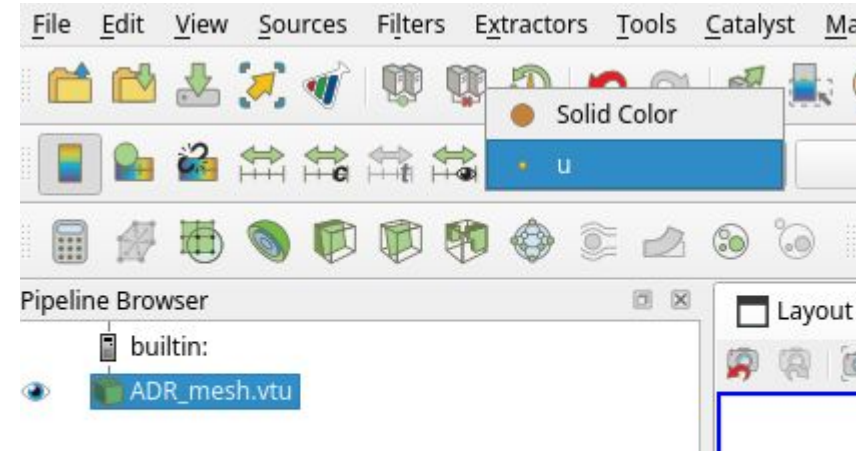
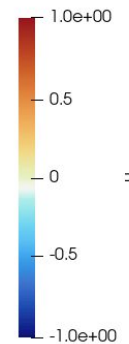
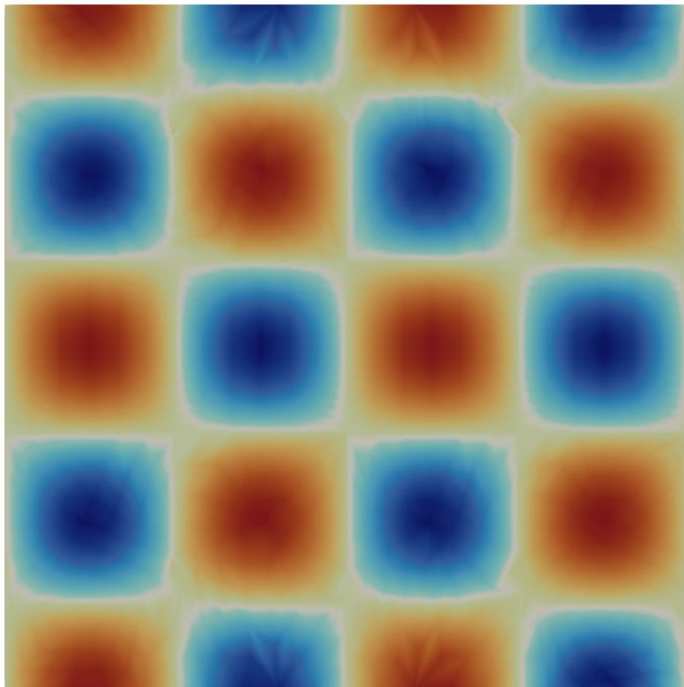
This will create a series of .chk files containing the solution at different time-steps, as specified in the session file and a final .fld containing the solution at  $t = t_{final}$ .

2. We convert the .fld to .vtu in order to visualize it with paraview.

```
> FieldConvert -f ADR_mesh.xml ADR_conditions.xml ADR_mesh.fld ADR_mesh.vtu  
Writing: "ADR_mesh.vtu"  
Written file: ADR_mesh.vtu
```

# Visualize in Paraview

1. Open Paraview
2. File -> Open -> /path/to/ADR\_mesh.vtu
3. Change plot variable



Be careful! This is a linear representation of the solution!

# Additional features: Restarting a simulation

Initial conditions from a prescribed expression:

```
<FUNCTION NAME="InitialConditions">  
  <E VAR="u" VALUE="Uinf * (y>0)" />  
  <E VAR="v" VALUE="0.5*Uinf*C/(Re_deltaStar_le * sqrt(1+x*C^2/Re_deltaStar_le))* (y>0)" />  
  <E VAR="p" VALUE="0" />  
</FUNCTION>
```

Initial conditions from previous solution:

```
<FUNCTION NAME="InitialConditions">  
  <F VAR="u,v,p" FILE="initialCond.fld" />  
</FUNCTION>
```

The name of restart file here is "initialCond.fld"

We can use any of the check point files ".chk"

We can rename the file as we want:  
Solution.chk -> restartfile.rst

And use the "restartfile.rst"

# Incompressible NS solver: *Solver info*

## Solver info

## Navier-Stokes equation

We can set the TimeIntegration method separately

There are two types of SVV here.

One for Spectral/HP (DGKernel)

If we have Quasi-3D simulation we can set SVV for the homogenous length too

For Quasi-3D simulations we need to activate FFT and HOMOGENOUS too

```
<SOLVERINFO>
<I  PROPERTY="SolverType"          VALUE="VelocityCorrectionScheme" />
<I  PROPERTY="EQTYPE"              VALUE="UnsteadyNavierStokes" />
<I  PROPERTY="EvolutionOperator"    VALUE="Nonlinear" />
<I  PROPERTY="Projection"          VALUE="Galerkin" />
<I  PROPERTY="GlobalSysSoln"        VALUE="DirectStaticCond" />
<I  PROPERTY="TimeIntegrationMethod" VALUE="IMEXOrder2" />
<I  PROPERTY="Driver"               VALUE="Standard" />
<I  PROPERTY="SpectralVanishingViscosity" VALUE="DGKernel" />
<I  PROPERTY="SpectralVanishingViscosityHomo1D" VALUE="ExpKernel" />
<I  PROPERTY="SPECTRALHPDEALIASING" VALUE="True" />
<I  PROPERTY="USEFFT"               VALUE="FFTW" />
<I  PROPERTY="HOMOGENEOUS"          VALUE="1D" />
<I  PROPERTY="DEALIASING"           VALUE="True" />
</SOLVERINFO>
```

# Incompressible NS solver: *Conditions*

## <PARAMETERS>

```

<P> TimeStep           = 0.025           </P>
<P> NumSteps           = 200             </P>
<P> IO_CheckSteps      = NumSteps/5       </P>
<P> IO_InfoSteps       = 1               </P>
<P> IO_FiltersInfoSteps = 100            </P>
<P> IO_CFLSteps        = 10             </P>
<!-- Reynolds number -->
<P> Re                 = 60              </P>
<P> Kinvis              = 1/Re           </P>
<!-- SVV parameters -->
<P> SVVCutoffRatio      = 0.5            </P>
<P> SVVCutoffRatioHomo1D = 0.5          </P>
<P> SVVDiffCoeffHomo1D  = 0.1           </P>

<!-- quasi-3D parameters -->
<P> HomModesZ           = 4              </P>
<P> LZ                  = 2*PI           </P>
</PARAMETERS>

```

## Parameters

For the **Incompressible Navier-Stokes** equations:

Kinvs : sets the kinematic viscosity

Density is 1 by default

So based you can set Kinvs using:

$Kinvs = U L / Re$

More control over the SVV and GJP stabilisations:

SVVCutoffRatio

SVVDiffCoeff

GJPJumpScale

For Quasi 3D simulations

HomModesZ : Number of Fourier planes in the z direction

LZ : Length of homogenous (spanwise) extrusion

# Adding filters!

- A variety of filters are supported in Nektar++ that complement the actual simulations.

|        |                                       |    |
|--------|---------------------------------------|----|
| 3.5    | Filters                               | 48 |
| 3.5.1  | Phase sampling                        | 49 |
| 3.5.2  | Aerodynamic forces                    | 49 |
| 3.5.3  | Benchmark                             | 51 |
| 3.5.4  | Cell history points                   | 52 |
| 3.5.5  | Checkpoint cell model                 | 52 |
| 3.5.6  | Checkpoint fields                     | 53 |
| 3.5.7  | Electrogram                           | 54 |
| 3.5.8  | Offset Phase                          | 54 |
| 3.5.9  | Hilbert Transform Phase based on FFTW | 55 |
| 3.5.10 | Error                                 | 56 |
| 3.5.11 | Integral                              | 57 |
| 3.5.12 | Integral                              | 57 |
| 3.5.13 | FieldConvert checkpoints              | 58 |
| 3.5.14 | History points                        | 59 |
| 3.5.15 | Kinetic energy and enstrophy          | 61 |
| 3.5.16 | Mean values                           | 62 |
| 3.5.17 | Modal energy                          | 62 |
| 3.5.18 | Moving body                           | 63 |
| 3.5.19 | Moving average of fields              | 63 |
| 3.5.20 | One-dimensional energy                | 64 |
| 3.5.21 | Reynolds stresses                     | 64 |
| 3.5.22 | Time-averaged fields                  | 65 |
| 3.5.23 | ThresholdMax                          | 66 |
| 3.5.24 | ThresholdMin value                    | 66 |
| 3.5.25 | Maximun/minimum fields                | 66 |
| 3.5.26 | Python script                         | 67 |
| 3.5.27 | Body-fitted velocity fields           | 69 |
| 3.5.28 | Lagrangian points Tracking            | 71 |

# Adding filters!

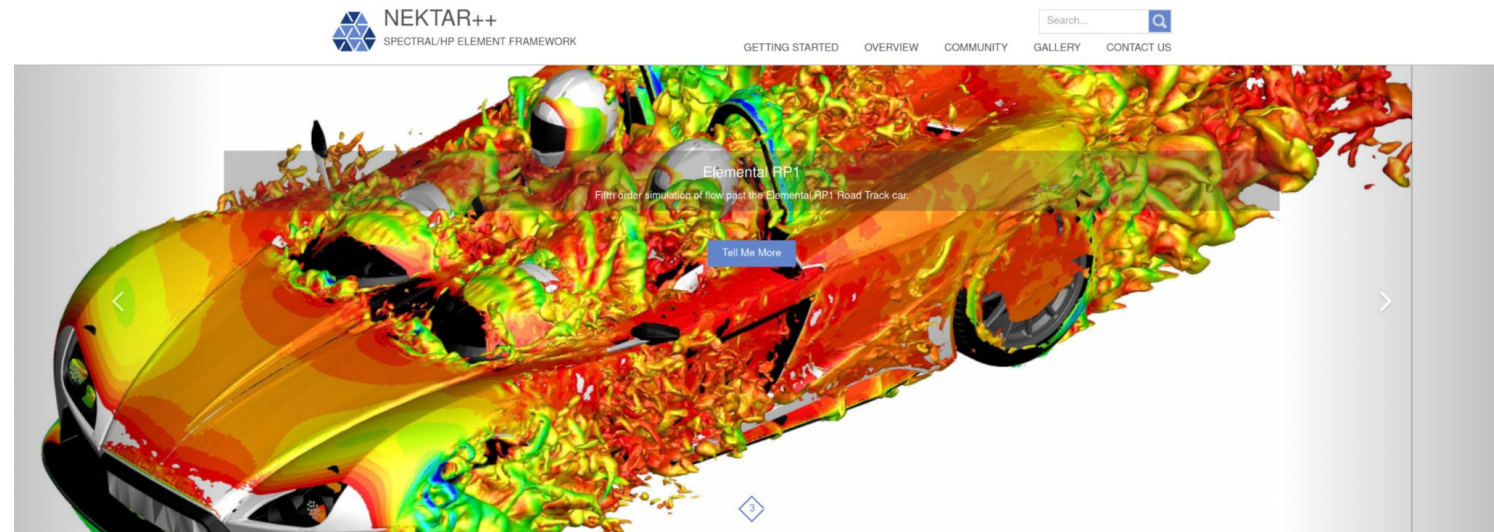
- Here we include the **ModalEnergy** and **HistoryPoints** filters.

```
6 <NEKTAR>
5 <EXPANSIONS>
4 |
3 </EXPANSIONS>
2 <CONDITIONS>
1 |
220 </CONDITIONS>
1 <FILTERS>
2 |
3 <FILTER TYPE="ModalEnergy">
4 | <PARAM NAME="OutputFile">EnergyFile</PARAM>
5 | <PARAM NAME="OutputFrequency">EnergySteps</PARAM>
6 </FILTER>
7
8 <FILTER TYPE="HistoryPoints">
9 | <PARAM NAME="OutputFile">HistoryPoints</PARAM>
10 | <PARAM NAME="OutputFrequency">HistorySteps</PARAM>
11 | <PARAM NAME="Points">
12 | -100 1 0
13 | -75 1 0
14 | -50 1 0
15 | ...
16 | 960 1 0
17 | 985 1 0
18 | 1010 1 0
19 | </PARAM>
20 </FILTER>
21 </FILTERS>
22 </NEKTAR>
```

# Summary: Nektar++ website

<https://www.nektar.info/>

- User guide
- Introductory video
- Blogs
- Tutorials
- Publications
- ...



What is Nektar++?



Latest Release



[gitlab.nektar.info](https://gitlab.nektar.info)





# Thanks!

# Questions?

